

Lab 1: Iterative Integer Multiplier

1 Introduction

The purpose of Lab 1 was to gain experience implementing hardware using Verilog and to practice design patterns such as datapath/control decomposition, FSMs, and streaming interfaces. We were required to implement two iterative integer multipliers: a fixed-latency baseline design and a variable-latency alternative. Both designs use a val/rdy stream interface and shift-and-add multiplication, but the alternative design exploits runs of zeros in the multiplier operand to skip cycles and reduce execution time. This lab builds the foundation for future labs that require progressively more complex RTL designs.

2 Alternative Design

The alternative design, with its datapath shown in Figure 1, represents our implementation of a **variable-latency iterative multiplier** that improves on the **baseline multiplier** by reducing the number of unnecessary cycles and terminating computation earlier. These optimizations are achieved with two primary mechanisms: **variable shifts** using the result from a **priority encoder**, and **early termination** using a **done flag**. Together, these changes allow for fewer overall cycle counts for most sets of multiplication, making the design more efficient than the baseline.

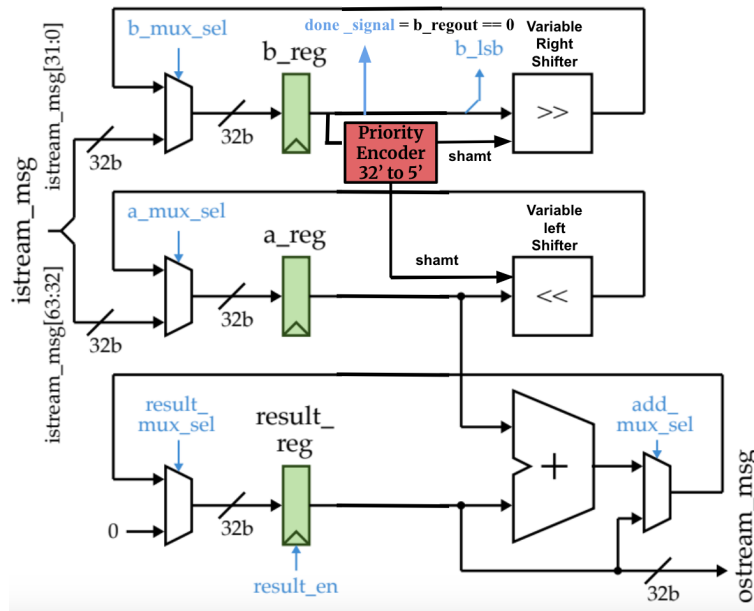


Figure 1: Alternative Design Datapath

In our baseline design, the datapath shifts the values stored in **a_reg** and **b_reg** by exactly one bit per cycle. Each cycle, the control unit checks the **least significant bit (LSB)** of **b_reg** and if it is 1, the datapath adds the value in **a_reg** to the **result_reg**, but if it is 0,

the datapath performs only the shift, wasting a cycle without making progress toward the final product. This fixed one-bit shifting results in many redundant cycles, especially when multiple consecutive zeros appear in `b_reg`.

To address this inefficiency, we instantiated a **priority encoder** in our alternative design to detect the number of consecutive zeros starting from the LSB of `b_reg`. Rather than shifting one bit at a time, the datapath shifts both `a_reg` and `b_reg` by the number of trailing zeros in a single cycle. This makes sure that each cycle either performs an addition or meaningfully advances toward the final result. For example, if `b_reg` contains the binary value `...1010000`, the encoder outputs a shift amount of `4`, so that `b_reg` is right-shifted by four bits, skipping directly to the next 1. This will allow our design to greatly reduce the number of wasted cycles. Our initial idea was to scan `b_reg` bit-by-bit to find the first 1, but such iterative constructs are not synthesizable in hardware, since we were trying to use a for/while loop. Instead, we implemented the **priority encoder** as a combinational module that checks all bits in parallel, outputting the position of the least significant 1. The result from the priority encoder provides the necessary number of shifts to make in order to skip consecutive zeros. This result is inputted into our original shifters in our baseline design, allowing them to become **variable shifters** as we can shift by different amounts.

The second inefficiency in the baseline multiplier was its fixed latency. Regardless of the operands, the baseline design always runs for **32–35 cycles**. In practice, however, the result is complete much earlier if `b_reg` becomes zero before all bits are processed. To optimize for this, the alternative design introduces a `done_signal`, which is asserted when the contents of `b_reg` are equal to zero. When this condition is true, the control unit immediately transitions to the **DONE state** from the **CALC state**, terminating the multiplication early. This can allow our hardware to not waste cycles once the correct result is available.

Together, these optimizations make the alternative design resemble a finite state machine (**FSM**) with a variable execution time. The priority encoder reduces the number of redundant shift cycles, while the **done flag** allows the multiplier to terminate as soon as possible. Both changes are lightweight in terms of hardware overhead but significantly improve performance compared to the baseline multiplier.

3 Testing Strategy

Our testing strategy combined directed tests, randomized tests, and reference-model checking to ensure functional correctness and robustness across both typical and corner-case scenarios. We specifically designed our testing process to build confidence in correctness by layering simple cases first and progressively moving toward stress tests and randomized verification.

Directed testing We began with directed tests since they provide the clearest insight into correctness and debugging. The goal was to validate that the design correctly handles fundamental operations before scaling to more complex cases. These included:

- *Zeros and Ones:* Multiplication with zero should always yield zero, while multiplication with one or negative one checks basic signed handling.
- *Small integers:* Examples such as 2×3 and -3×5 confirm correct addition and subtraction behavior within the iterative multiplier.

- *Large magnitudes*: Inputs close to 16-bit and 32-bit boundaries (65535×65535) verified truncation and overflow behavior under modulo- 2^{32} semantics.
- *Structured bit patterns*: Masked and sparse operands highlighted whether the variable-latency design correctly skips over runs of zeros.
- *Corner cases*: Situations like `INT_MIN * -1` stress-test two’s complement arithmetic and overflow behavior.

Randomized testing. After establishing baseline correctness, we moved to randomized tests to explore a much broader input space than we could manually design. Operand pairs were generated using fixed random seeds to ensure reproducibility. Each test compared DUT results with a Python functional-level reference model, truncated to 32 bits. Randomized testing is especially valuable for uncovering unexpected interactions in the FSM or datapath that directed tests might not cover.

Val/rdy stress testing. Since both designs use latency-insensitive val/rdy interfaces, it was important to verify that handshake correctness was maintained even under stalls. We inserted delays on the source and sink streams independently, and in combination, to simulate realistic pipeline stalls. This ensures that the DUT never drops, duplicates, or misaligns data across interface boundaries.

Evidence of correctness. The combination of directed and randomized tests, along with stall inclusion, produced consistent matches with the reference model across thousands of test cases. Directed testing gave confidence in corner-case correctness, while randomized testing provided statistical assurance. Stress testing of the interface confirmed proper val/rdy behavior independent of operand values.

Table 1 summarizes our directed test categories.

Table 1: Summary of directed test categories.

Category	Examples
Zeros/Ones	$0 \times N$, $1 \times N$, $-1 \times N$
Small values	-3×5 , 7×-8
Large values	65535×65535
Masked patterns	$0x12340000 \times 0x00020000$
Sparse operands	9×131073
Dense operands	$0xFFFFFFFF \times 0xFFFFFFFF$
Corner cases	$INT_MIN \times -1$

Together, these strategies provided a comprehensive and balanced testing approach, demonstrating that the designs were functionally correct, robust to corner cases, and compliant with the val/rdy protocol.

4 Evaluation

We evaluated the two multiplier designs in terms of performance, area, energy, and cycle time to quantify the benefits and tradeoffs of the alternative, variable-latency design.

Performance. Both designs were tested against a set of input patterns, ranging from highly optimized cases with long runs of zeros to minimally optimized, dense inputs such as a multiplication of -1×-1 . According to Figure 2, the alternative design offers minimal improvement for more dense inputs, since its optimizations rely on skipping cycles based on the amount of consecutive zeros. For example, the small negative $\times$ negative test requires nearly the same number of cycles as the baseline (34.49 vs. 35.09 cycles, a $1.02\times$ speedup). Similarly, small positive $\times$ negative inputs show minimal gain. In contrast, as shown in Figure 2, inputs with many zeros or small positive values benefit substantially. For instance, the small positive $\times$ positive set completes in only 6.30 cycles on average, which is just 18% of the baseline (35.18 cycles), yielding a $5.58\times$ speedup. Test cases dominated by zeros or ones also see dramatic improvements, requiring as few as 6.78 cycles per multiplication ($\sim 19\%$ of baseline). Sparse patterns similarly average under 9 cycles, about 25% of the baseline. Even large positive $\times$ positive values improve significantly, cutting cycles by more than half. Across a randomized input set spanning all possible values, the alternative design lowers the average number of cycles per multiplication to 19.28, approximately 55% of the baseline's average of 35.18, demonstrating substantial throughput improvement and highlights how effective variable-latency optimization is.

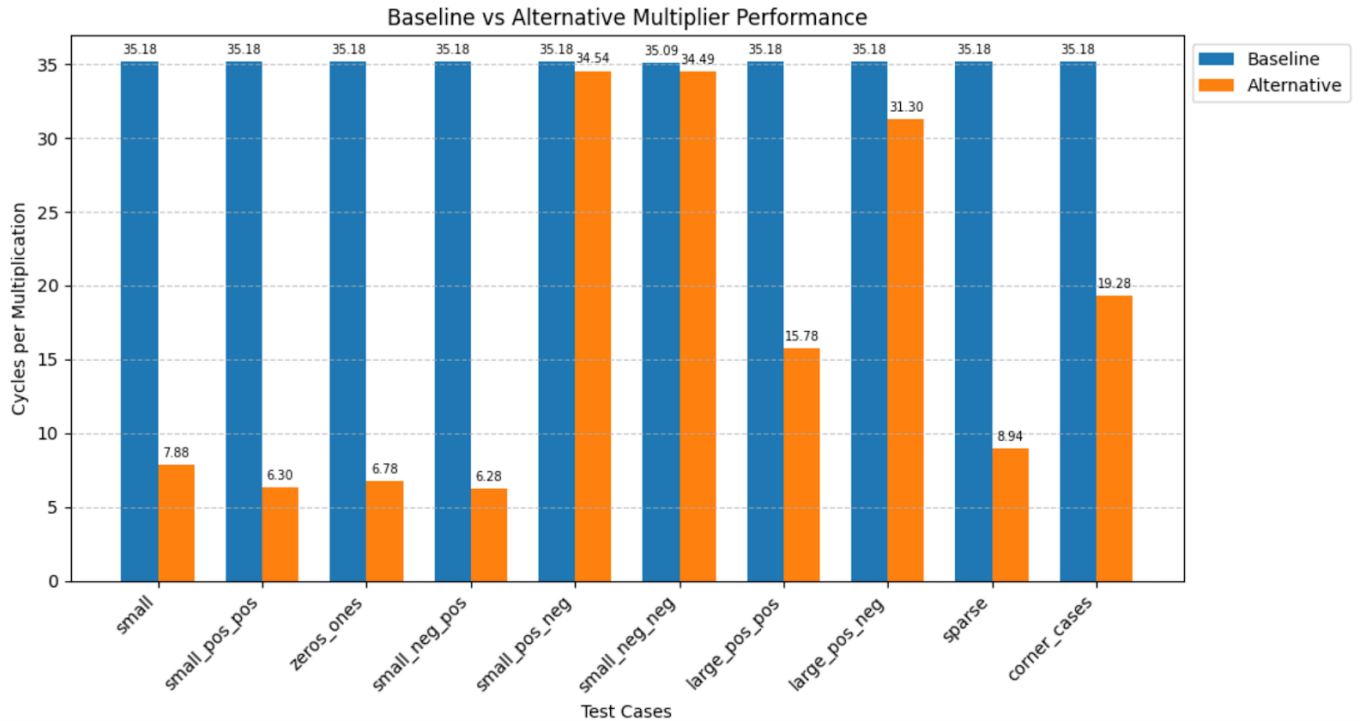


Figure 2: Performance Comparisons between Baseline and Alternative Design

Area. The alternative design requires additional logic since it uses a priority encoder and 5-bit variable shifters, which slightly increase combinational complexity compared to the baseline which uses 1-bit constant shifters. Nevertheless, the overall structural simplicity is maintained, including a single adder, three registers, muxes, and shifters. The slight area increase is justified by the substantial performance gains, making the tradeoff highly favorable for most applications.

Energy. Although the priority encoder and wider variable shifters toggle between more gates per cycle, increasing energy consumption slightly for dense inputs, the overall reduction in total cycles generally offsets this overhead. For sparse input patterns, the alternative design can even consume less energy than the baseline. This demonstrates that the alternative design improves energy efficiency in several practical scenarios without compromising much performance.

Cycle Time. In the baseline design, the critical path passes through the adder. In the alternative design, the critical path includes the priority encoder and wider shifters, resulting in a slightly longer propagation delay requiring a slight reduction in maximum clock frequency. However, the reduction in cycles per multiplication more than compensates for this small increase in delay, yielding a net throughput improvement. The variable-latency approach provides a clear advantage in cycle efficiency without a significant penalty that is found in the clock speeds.

5 Conclusion

The baseline fixed-latency multiplier is simple and predictable, always taking 35 (35.18) cycles. The alternative variable-latency multiplier achieved better performance on inputs with sparse multipliers, often reducing execution time by multiple factors. The cost was slightly higher area and a modestly longer cycle time. On dense inputs, the speedup diminishes, showing that operand patterns significantly affect performance. If predictable latency is desired, the baseline design may be preferable, but for typical workloads where multipliers are not fully dense, the variable-latency design provides clear advantages.

Overall, this comparison highlights a tradeoff between simplicity and efficiency in hardware design. Our baseline design offers consistency and ease of verification, while the alternative exploits data-dependent behavior to speed up common cases. Applications prioritizing timing guarantees may lean toward the baseline, whereas performance-critical systems can benefit substantially from the variable-latency approach.

6 Work Distribution

Jamayne implemented the baseline multiplier datapath, FSM and wrote the directed test cases + some random test cases. Md implemented the alternative multiplier datapath, including the priority encoder and variable shifters, and added tests to the simulation files to evaluate the two designs. Both students collaborated on writing the report. Generative AI tools were used for improving code comments, clarifying Verilog syntax, and improving lab report grammar, but all final designs and analysis were reviewed and understood by both of us.

7 Appendix

Table 2: Cycle counts and cycles per multiplication for baseline and alternative multipliers.

Test Case	Baseline total cycles (per mult)	Alternative total cycles (per mult)	Speedup
small	1759 (35.18)	394 (7.88)	4.47×
small_pos_pos	1759 (35.18)	315 (6.30)	5.58×
zeros_ones	1759 (35.18)	339 (6.78)	5.18×
small_neg_pos	1759 (35.18)	314 (6.28)	5.60×
small_pos_neg	1759 (35.18)	1727 (34.54)	1.02×
small_neg_neg	3509 (35.09)	3449 (34.49)	1.02×
large_pos_pos	1759 (35.18)	789 (15.78)	2.23×
large_pos_neg	1759 (35.18)	1565 (31.30)	1.12×
sparse	1759 (35.18)	447 (8.94)	3.94×
corner_cases	1759 (35.18)	964 (19.28)	1.82×